EXPERIMENT 1 — Linux VM Installation and System Configuration

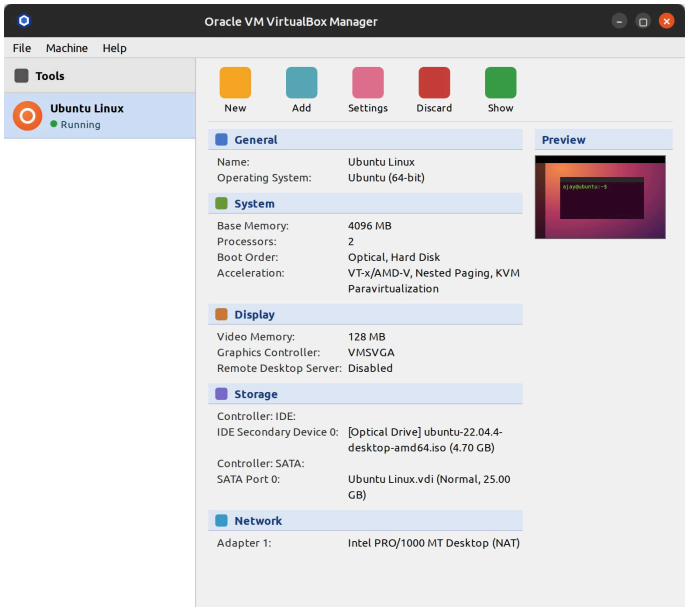
AIM

To install Linux in a virtual machine and perform basic system configuration and Linux commands.

PROCEDURE

1. Open Oracle VirtualBox on the computer.
2. Click New to create a virtual machine.
3. Enter the name as `Ubuntu Linux`.
4. Select the Linux ISO file.
5. Select Linux as the operating system and Ubuntu as the version.
6. Allocate RAM and CPU resources.
7. Create a virtual hard disk and allocate storage.
8. Click Finish and select the created VM.
9. Click Start and install Ubuntu by following the installation instructions.

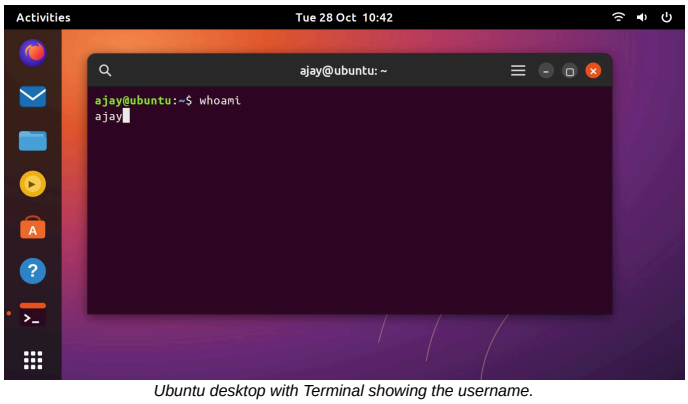
SCREENSHOT 1



VirtualBox showing the Ubuntu VM configuration.

10. Enter the username and password during installation.
11. Complete installation and restart the virtual machine.
12. Open the Ubuntu Terminal.
13. Type `whoami` and press Enter to display the current username.

SCREENSHOT 2

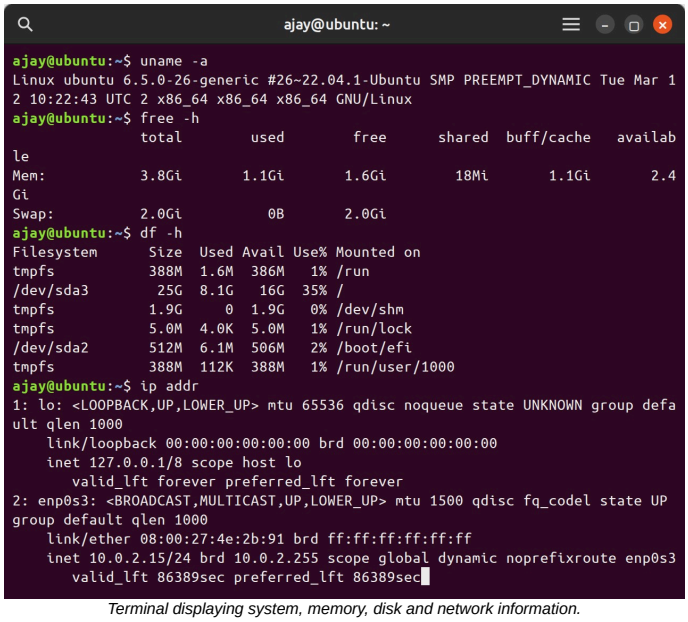


Ubuntu desktop with Terminal showing the username.

14. Execute the following commands one by one:

```
uname -a
free -h
df -h
ip addr
```

SCREENSHOT 3



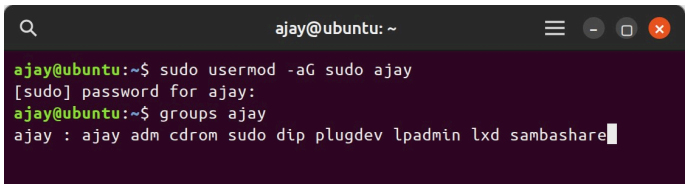
Terminal displaying system, memory, disk and network information.

15. Open Terminal and execute:

```
sudo usermod -aG sudo username
```

Replace `username` with the actual username.

SCREENSHOT 4



Terminal showing the user configuration command.

RESULT

Linux VM was installed and basic system configuration and commands were performed successfully.

EXPERIMENT 2 — Linux Package Installation and Service Management

AIM

To install Linux packages and manage system services using Linux commands.

PROCEDURE

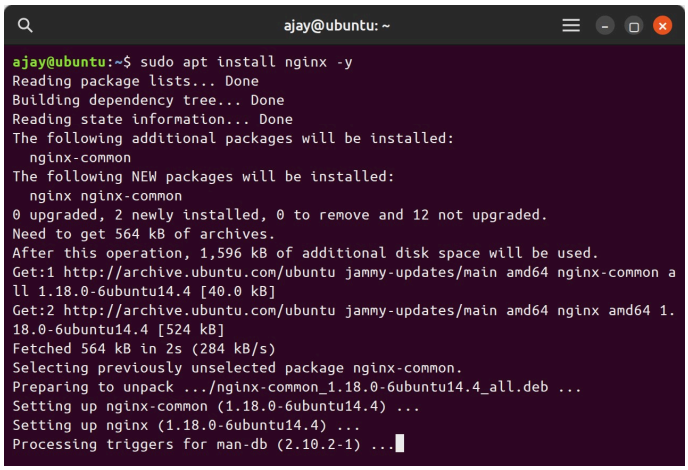
1. Start the Ubuntu virtual machine.
2. Log in using the Linux username and password.
3. Open the Terminal application.
4. Update the package repository:

```
sudo apt update
```

5. Enter the password when prompted.
6. Wait until the package update completes.
7. Install the Nginx web server:

```
sudo apt install nginx -y
```

SCREENSHOT 1



Terminal showing Nginx package installation.

8. Wait until installation finishes.
9. Start the Nginx service:

```
sudo systemctl start nginx
```

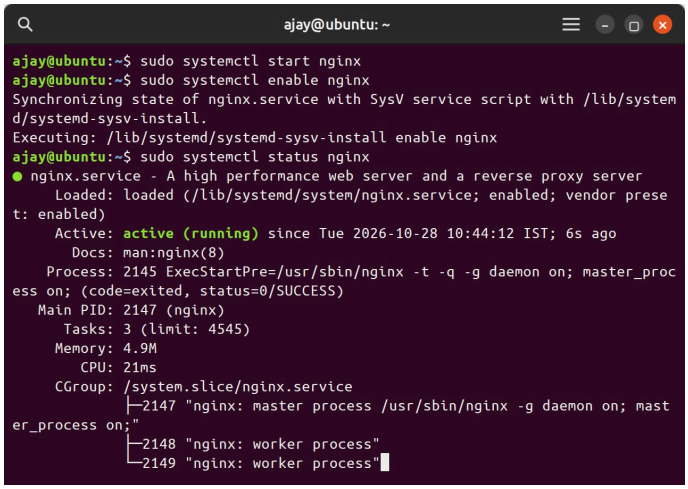
10. Enable Nginx to start automatically:

```
sudo systemctl enable nginx
```

11. Check the service status:

```
sudo systemctl status nginx
```

SCREENSHOT 2



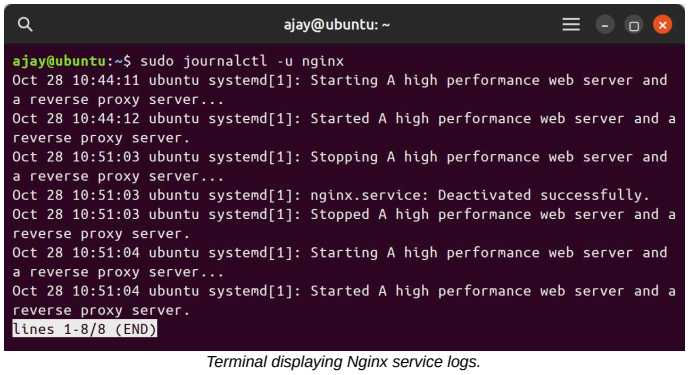
Terminal showing Nginx active and running.

12. Open Terminal and execute:

```
sudo journalctl -u nginx
```

13. Observe the service logs.

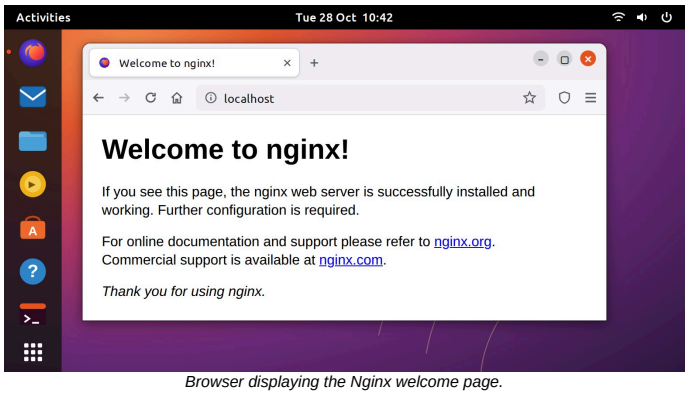
SCREENSHOT 3



Terminal displaying Nginx service logs.

14. Open Firefox in Ubuntu.
15. Enter `http://localhost` in the address bar.

SCREENSHOT 4



Browser displaying the Nginx welcome page.

RESULT

Linux packages were installed and the Nginx service was started, enabled and verified successfully.

EXPERIMENT 3 — SSH Key-Based Authentication

AIM

To configure SSH key-based authentication between two Linux systems.

PROCEDURE

1. Start the first Linux VM.
2. Start the second Linux VM.
3. Log in to both systems.
4. Open Terminal in the first VM.
5. Update the package repository:

```
sudo apt update
```

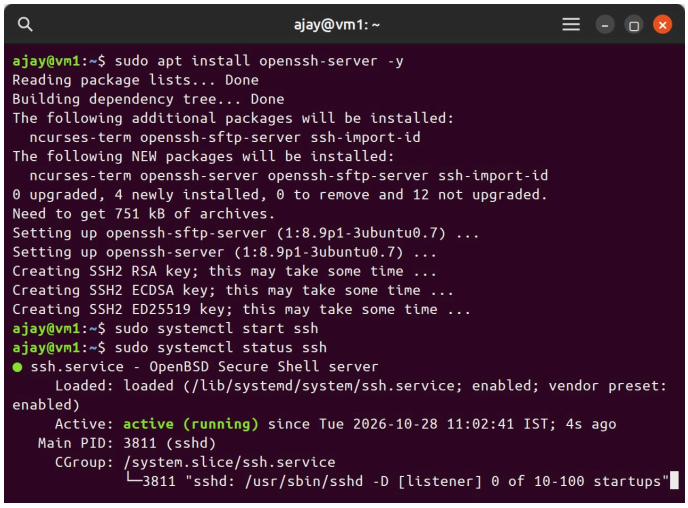
6. Install the SSH server:

```
sudo apt install openssh-server -y
```

7. Start the SSH service:

```
sudo systemctl start ssh
```

SCREENSHOT 1



Terminal showing SSH server installation.

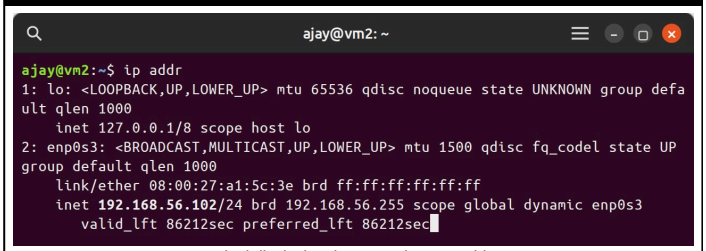
8. Check SSH service status:

```
sudo systemctl status ssh
```

9. Find the IP address of the second VM:

```
ip addr
```

SCREENSHOT 2



Terminal displaying the second VM IP address.

10. Open Terminal in the first VM.
11. Generate SSH keys:

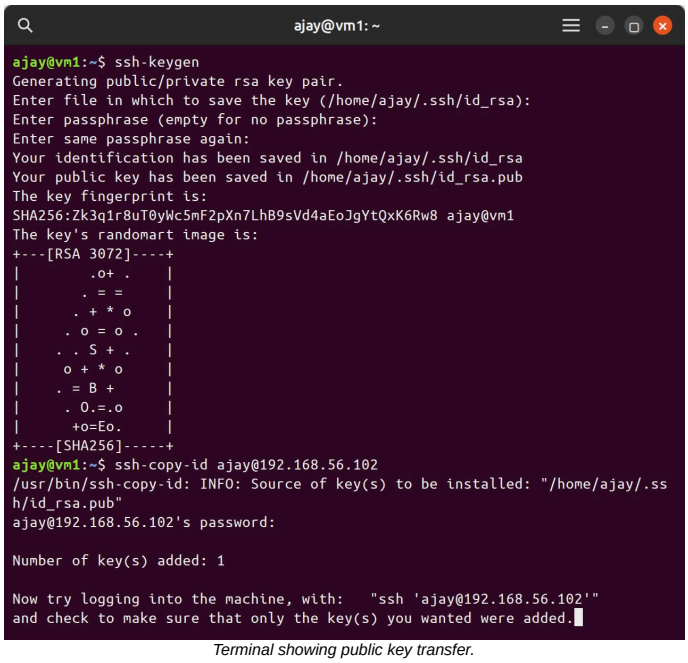
```
ssh-keygen
```

12. Press Enter to accept the default file location and prompts.
13. Copy the public key to the second VM:

```
ssh-copy-id username@IP
```

Replace the username and IP with the second VM's details.

SCREENSHOT 3



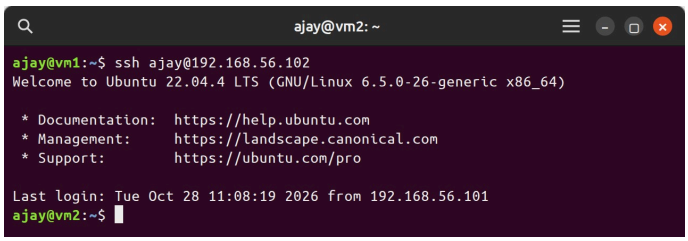
Terminal showing public key transfer.

14. Connect to the second VM:

```
ssh username@IP
```

15. Verify that the second VM's terminal prompt appears without asking for the account password.

SCREENSHOT 4



Terminal showing successful SSH login.

RESULT

SSH key-based authentication was configured and remote login was verified successfully.

EXPERIMENT 4 — Secure File Transfer Using SCP

AIM

To transfer files and directories securely between Linux systems using SCP.

PROCEDURE

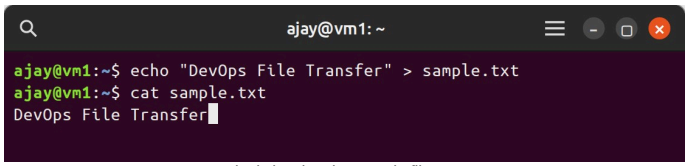
1. Start both Linux VMs.
2. Log in to the first VM.
3. Open Terminal.
4. Create a sample file:

```
echo "DevOps File Transfer" > sample.txt
```

5. Verify the file:

```
cat sample.txt
```

SCREENSHOT 1



Terminal showing the sample file content.

6. Find the IP address of the second VM:

```
ip addr
```

7. Ensure SSH is running on the second VM.
8. Transfer the file:

```
scp sample.txt username@IP:~
```

9. Enter the password if requested.
10. Wait until the transfer completes.

SCREENSHOT 2

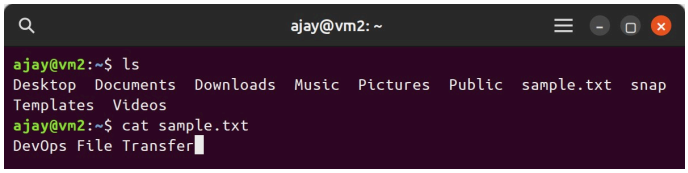


Terminal showing successful SCP file transfer.

11. Log in to the second VM.
12. Open Terminal.
13. Verify the transferred file:

```
ls
cat sample.txt
```

SCREENSHOT 3



Terminal displaying the received file.

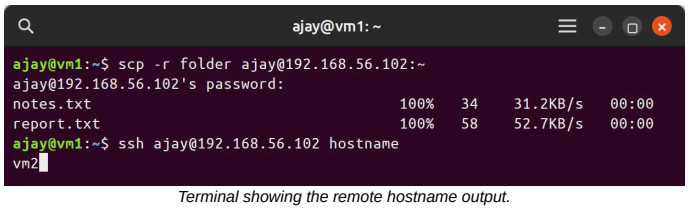
14. Return to the first VM and transfer a directory:

```
scp -r folder username@IP:~
```

15. Execute a remote command:

```
ssh username@IP hostname
```

SCREENSHOT 4



Terminal showing the remote hostname output.

RESULT

Files and directories were transferred securely between Linux systems using SCP.

EXPERIMENT 5 — Web Server Deployment Using Docker

AIM

To deploy an Nginx web server using Docker and verify its accessibility through a browser.

PROCEDURE

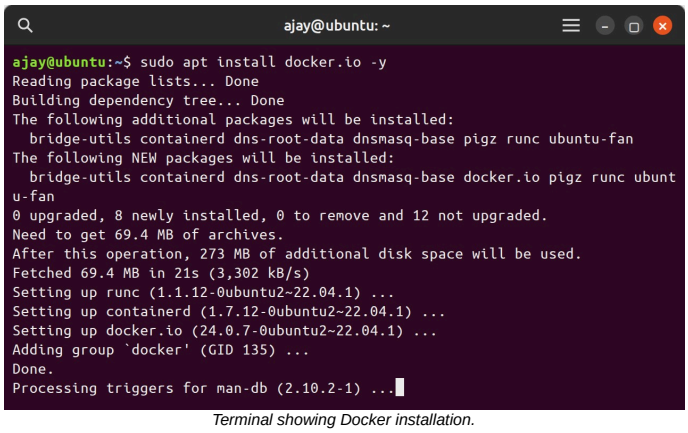
1. Start the Ubuntu VM.
2. Log in to Ubuntu.
3. Open Terminal.
4. Update the package repository:

```
sudo apt update
```

5. Install Docker:

```
sudo apt install docker.io -y
```

SCREENSHOT 1



Terminal showing Docker installation.

6. Start Docker:

```
sudo systemctl start docker
```

7. Enable Docker:

```
sudo systemctl enable docker
```

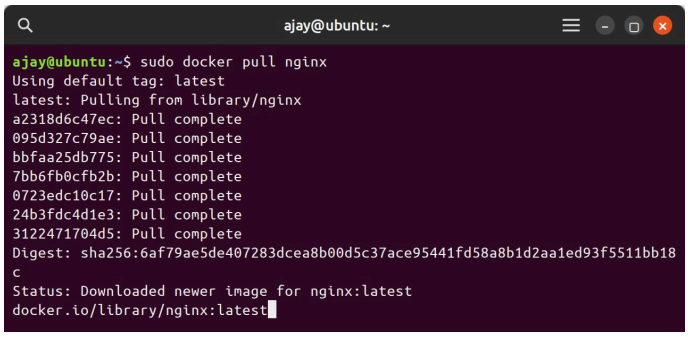
8. Check Docker status:

```
sudo systemctl status docker
```

9. Download the Nginx image:

```
sudo docker pull nginx
```

SCREENSHOT 2



Terminal showing the Nginx image download.

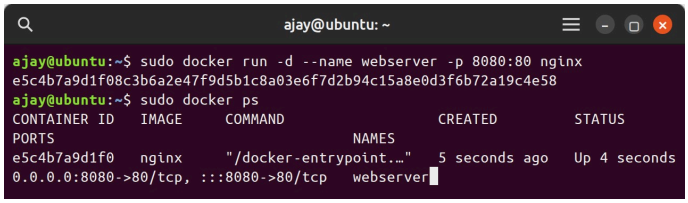
10. Create and start the container:

```
sudo docker run -d --name webserver -p 8080:80 nginx
```

11. Check running containers:

```
sudo docker ps
```

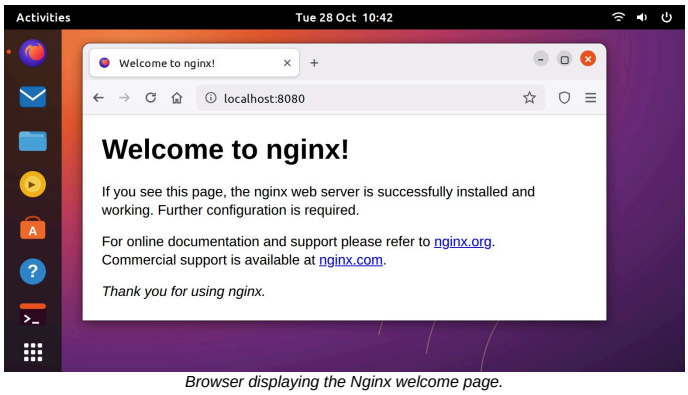
SCREENSHOT 3



Terminal showing the running Nginx container.

12. Open Firefox in Ubuntu.
13. Enter `http://localhost:8080`.
14. Wait for the Nginx welcome page to appear.

SCREENSHOT 4



Browser displaying the Nginx welcome page.

15. Stop the container:

```
sudo docker stop webserver
```

RESULT

The Nginx web server was deployed using Docker and accessed through a browser.

EXPERIMENT 6 — Container Image Creation and Management

AIM

To create a custom Docker image using a Dockerfile and perform image management operations.

PROCEDURE

1. Start Ubuntu.
2. Log in to the system.
3. Open Terminal.
4. Create a project directory:

```
mkdir myweb
```

5. Open the directory:

```
cd myweb
```

6. Create an HTML file:

```
echo "Welcome to Docker" > index.html
```

7. Create a Dockerfile:

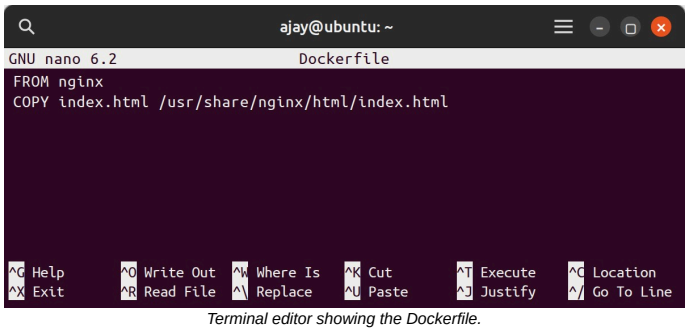
```
nano Dockerfile
```

8. Enter the following instructions:

```
FROM nginx
COPY index.html /usr/share/nginx/html/index.html
```

9. Save and close the file.

SCREENSHOT 1

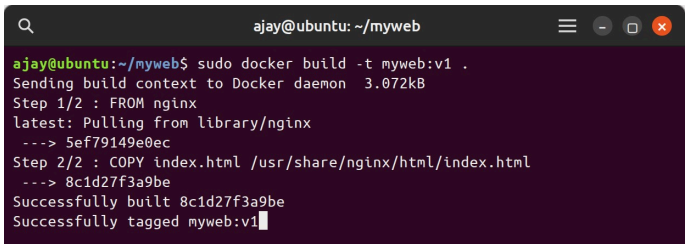


Terminal editor showing the Dockerfile.

10. Build the custom image:

```
sudo docker build -t myweb:v1 .
```

SCREENSHOT 2



Terminal showing successful image build.

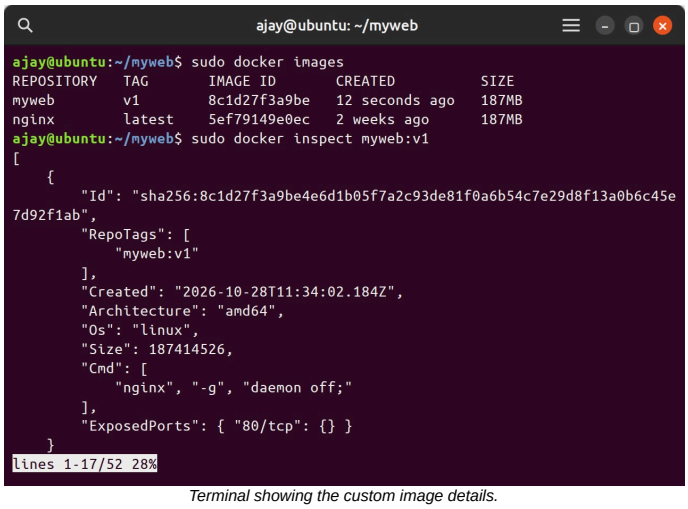
11. List Docker images:

```
sudo docker images
```

12. Inspect the custom image:

```
sudo docker inspect myweb:v1
```

SCREENSHOT 3



Terminal showing the custom image details.

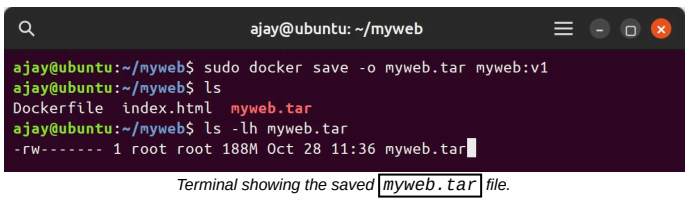
13. Save the image:

```
sudo docker save -o myweb.tar myweb:v1
```

14. Verify the saved file:

```
ls
```

SCREENSHOT 4



Terminal showing the saved `myweb.tar` file.

15. Load the saved image:

```
sudo docker load -i myweb.tar
```

RESULT

A custom Docker image was created, inspected, saved and loaded successfully.

EXPERIMENT 7 — Automated Web Server Configuration Using Puppet

AIM

To automate web server installation and service configuration using Puppet.

PROCEDURE

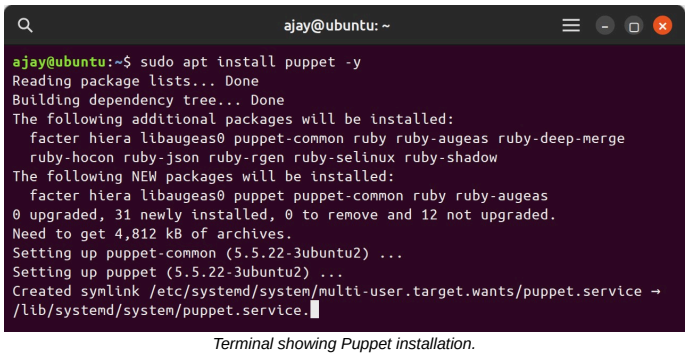
1. Start Ubuntu.
2. Log in to the system.
3. Open Terminal.
4. Update the package repository:

```
sudo apt update
```

5. Install Puppet:

```
sudo apt install puppet -y
```

SCREENSHOT 1



Terminal showing Puppet installation.

6. Create a Puppet manifest:

```
nano web.pp
```

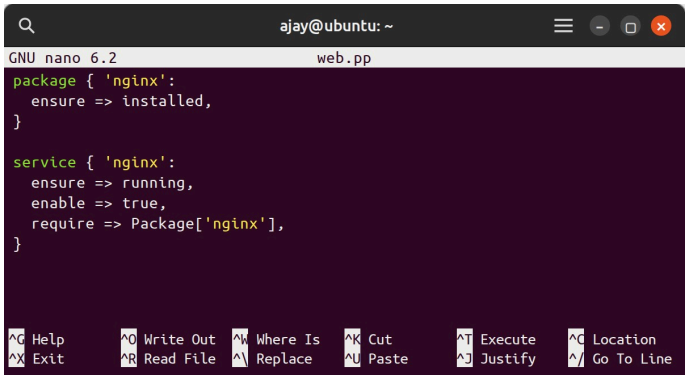
7. Enter the following code:

```
package { 'nginx':
  ensure => installed,
}

service { 'nginx':
  ensure => running,
  enable => true,
  require => Package['nginx'],
}
```

8. Save and close the file.

SCREENSHOT 2



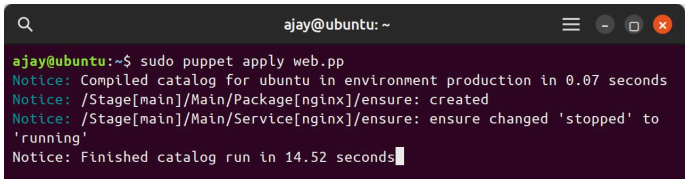
Terminal editor showing the Puppet manifest.

9. Apply the configuration:

```
sudo puppet apply web.pp
```

10. Wait until Puppet completes.
11. Observe the package and service configuration output.

SCREENSHOT 3



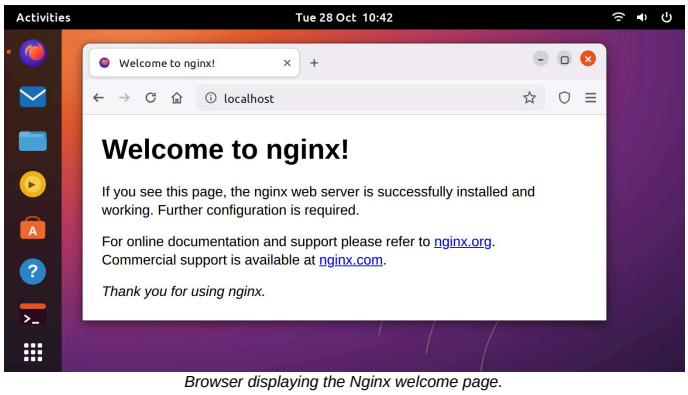
Terminal showing Puppet apply output.

12. Check the Nginx service:

```
sudo systemctl status nginx
```

13. Open Firefox.
14. Enter `http://localhost`.

SCREENSHOT 4



Browser displaying the Nginx welcome page.

15. Close the browser and Terminal.

RESULT

Puppet successfully automated Nginx installation and service configuration.

EXPERIMENT 8 — Automated Software Deployment Using Jenkins

AIM

To automate software deployment using a Jenkins Freestyle project.

PROCEDURE

1. Start Ubuntu.
2. Open Terminal.
3. Install Java:

```
sudo apt install openjdk-17-jdk -y
```

4. Install Jenkins using the configured Jenkins repository.
5. Start the Jenkins service:

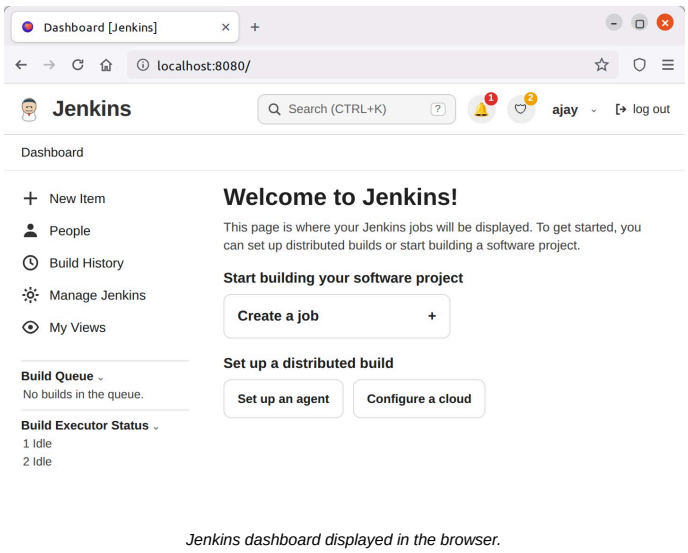
```
sudo systemctl start jenkins
```

6. Enable Jenkins:

```
sudo systemctl enable jenkins
```

7. Open Firefox.
8. Enter `http://localhost:8080`.
9. Unlock Jenkins and complete the initial setup.

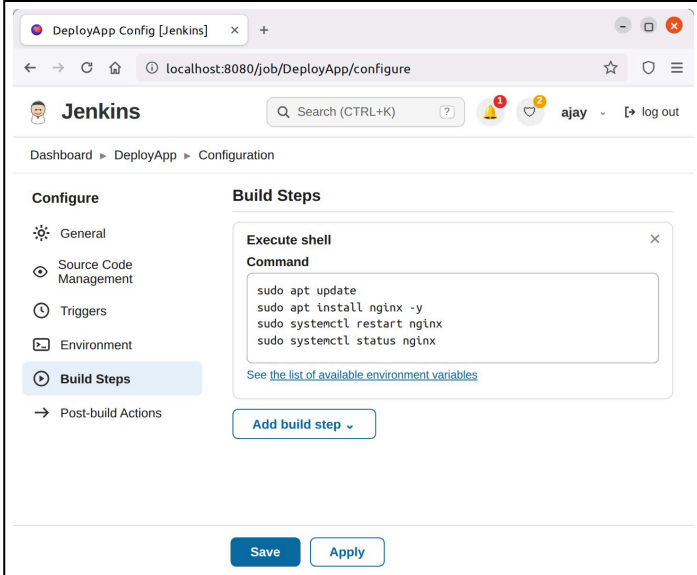
SCREENSHOT 1



Jenkins dashboard displayed in the browser.

10. Click New Item.
11. Enter the project name as `DeployApp`.
12. Select Freestyle project and click OK.

SCREENSHOT 2



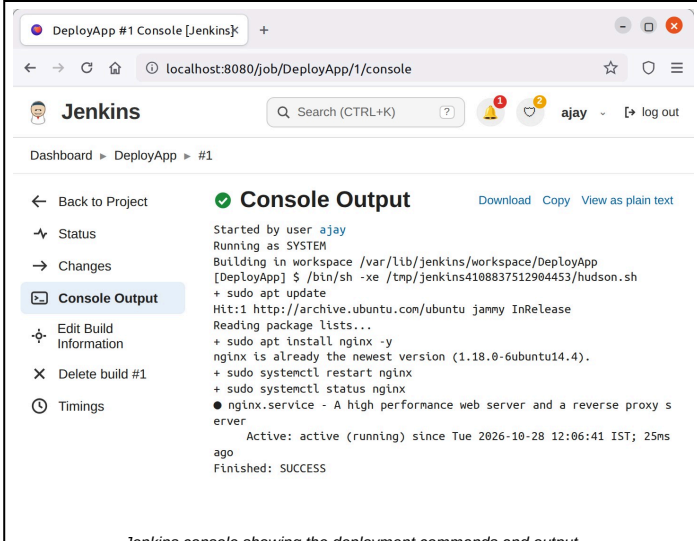
Jenkins Freestyle project configuration page.

13. Select Build Steps → Execute shell and enter:

```
sudo apt update
sudo apt install nginx -y
sudo systemctl restart nginx
sudo systemctl status nginx
```

14. Click Save, then click Build Now. Open the build's Console Output.

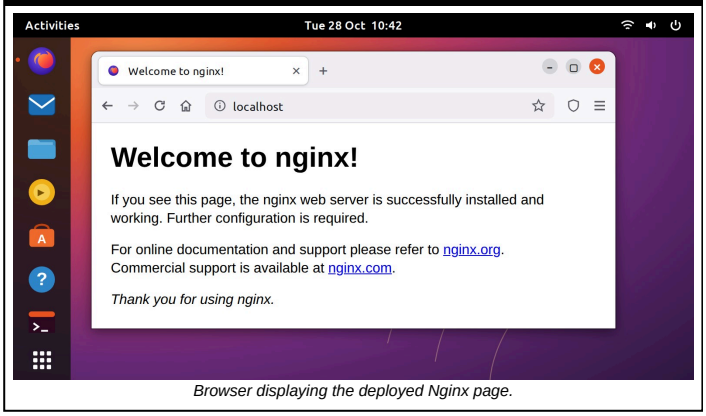
SCREENSHOT 3



Jenkins console showing the deployment commands and output.

15. Open Firefox and enter `http://localhost`.

SCREENSHOT 4



Browser displaying the deployed Nginx page.

RESULT

Software deployment was automated using a Jenkins Freestyle project.

Note: The Jenkins build user must have permission to execute the required `sudo` commands.

EXPERIMENT 9 — Automated Java Build Using Maven and Jenkins

AIM

To build and package a Java application using Maven through Jenkins.

PROCEDURE

1. Start Ubuntu.
2. Open Terminal.
3. Install Java:

```
sudo apt install openjdk-17-jdk -y
```

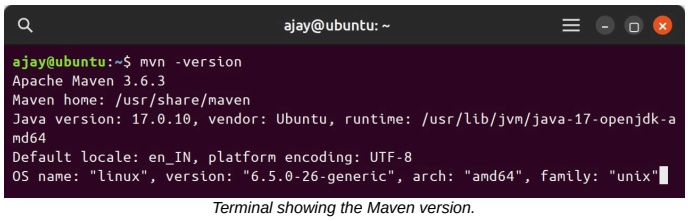
4. Install Maven:

```
sudo apt install maven -y
```

5. Verify Maven installation:

```
mvn -version
```

SCREENSHOT 1



Terminal showing the Maven version.

6. Create a project directory:

```
mkdir JavaApp
```

7. Open the project directory:

```
cd JavaApp
```

8. Create the Java source file:

```
nano App.java
```

9. Enter the following code:

```
public class App {
    public static void main(String[] args) {
        System.out.println("Hello DevOps");
    }
}
```

SCREENSHOT 2



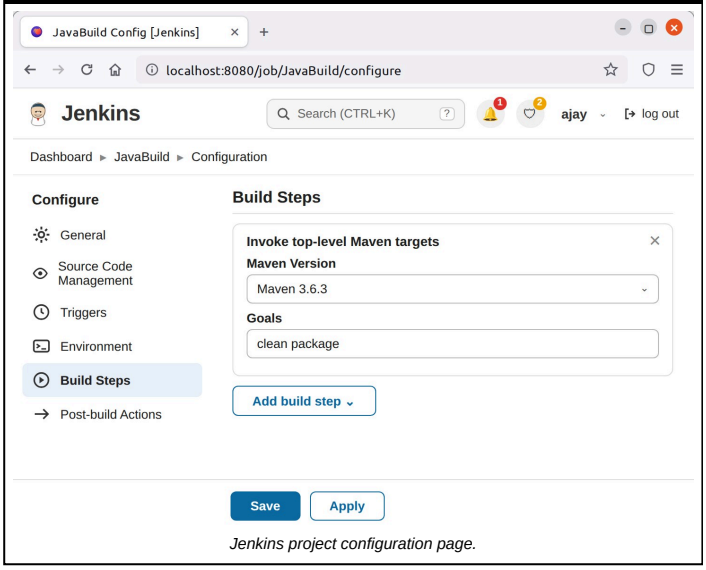
Terminal editor showing the Java program.

10. Create the Maven project file:

```
nano pom.xml
```

11. Enter the Maven project configuration and save the file.
12. Open Jenkins in Firefox.
13. Click New Item, enter `JavaBuild`, select Freestyle project, and click OK.

SCREENSHOT 3



Jenkins project configuration page.

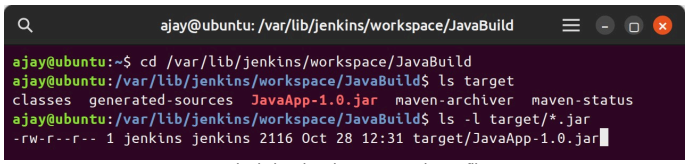
14. Configure the project to run Maven with the goals:

```
clean package
```

Save and build the project.

15. Open the project directory and verify the generated JAR file in the `target` folder.

SCREENSHOT 4



Terminal showing the generated JAR file.

RESULT

A Java application was compiled and packaged into a JAR file using Maven and Jenkins.

EXPERIMENT 10 — Continuous Integration Using Jenkins Pipeline

AIM

To implement a Jenkins Pipeline for automated software compilation, testing and packaging.

PROCEDURE

1. Start Ubuntu.
2. Open Terminal.
3. Install Java:

```
sudo apt install openjdk-17-jdk -y
```

4. Install Maven:

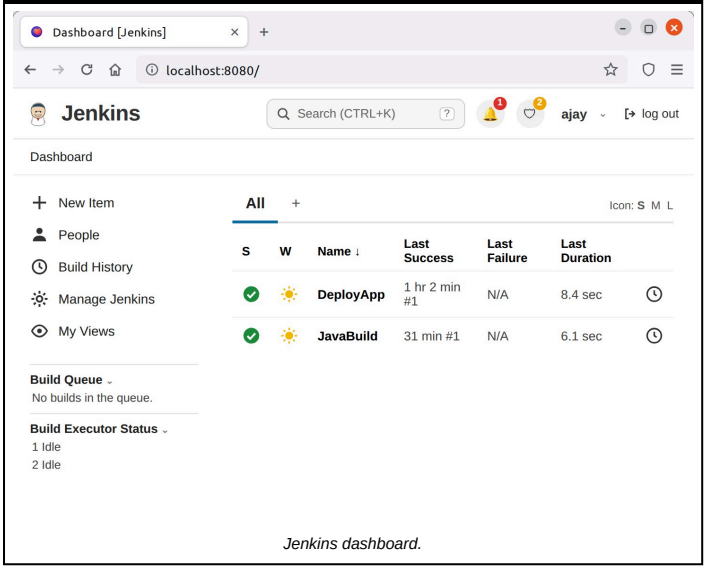
```
sudo apt install maven -y
```

5. Start Jenkins:

```
sudo systemctl start jenkins
```

6. Open Firefox.
7. Enter `http://localhost:8080`.
8. Log in to Jenkins.

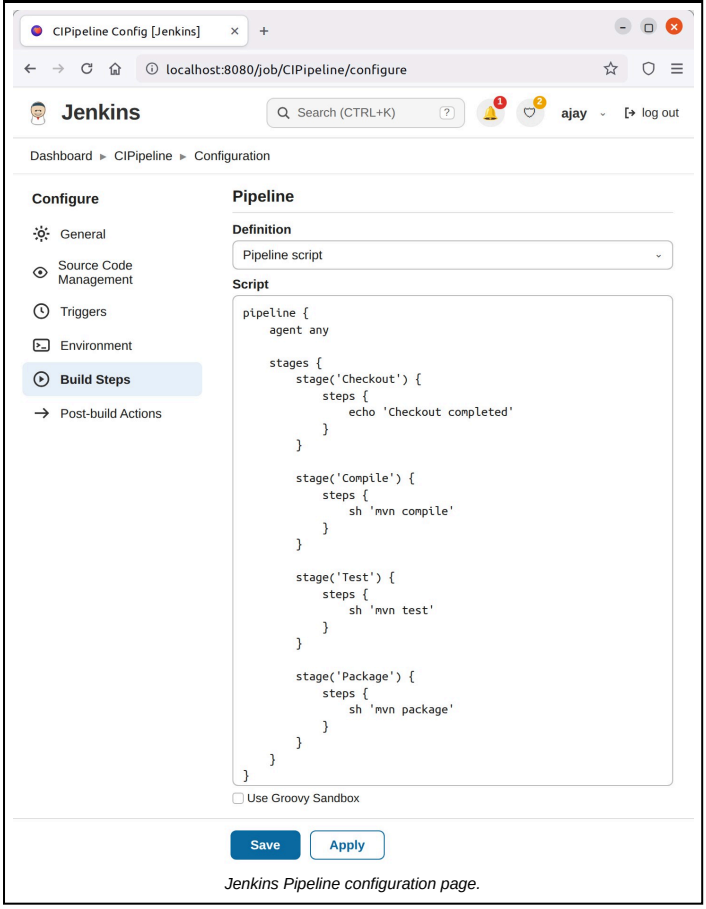
SCREENSHOT 1



Jenkins dashboard.

9. Click New Item.
10. Enter the project name as `CIPipeline`.
11. Select Pipeline and click OK.

SCREENSHOT 2



Jenkins Pipeline configuration page.

12. Select Pipeline → Definition → Pipeline script.
13. Enter the following pipeline:

```
pipeline {
    agent any

    stages {
        stage('Checkout') {
            steps {
                echo 'Checkout completed'
            }
        }

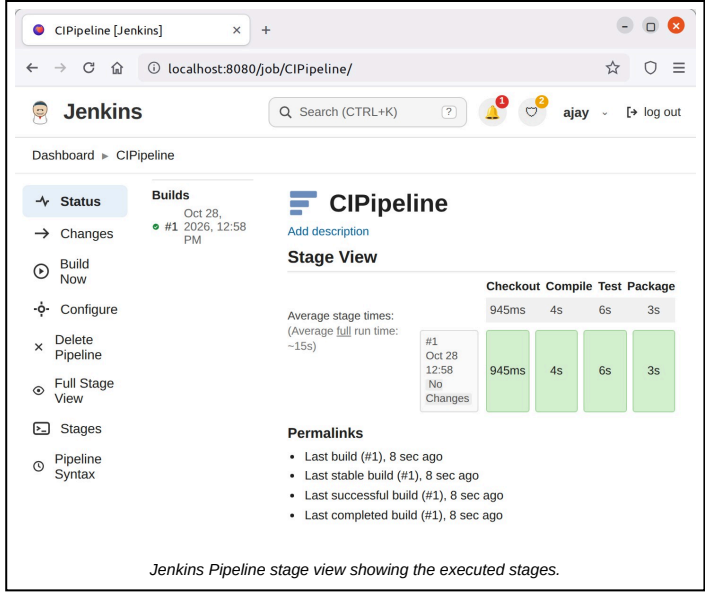
        stage('Compile') {
            steps {
                sh 'mvn compile'
            }
        }

        stage('Test') {
            steps {
                sh 'mvn test'
            }
        }

        stage('Package') {
            steps {
                sh 'mvn package'
            }
        }
    }
}
```

14. Click Save and then Build Now.

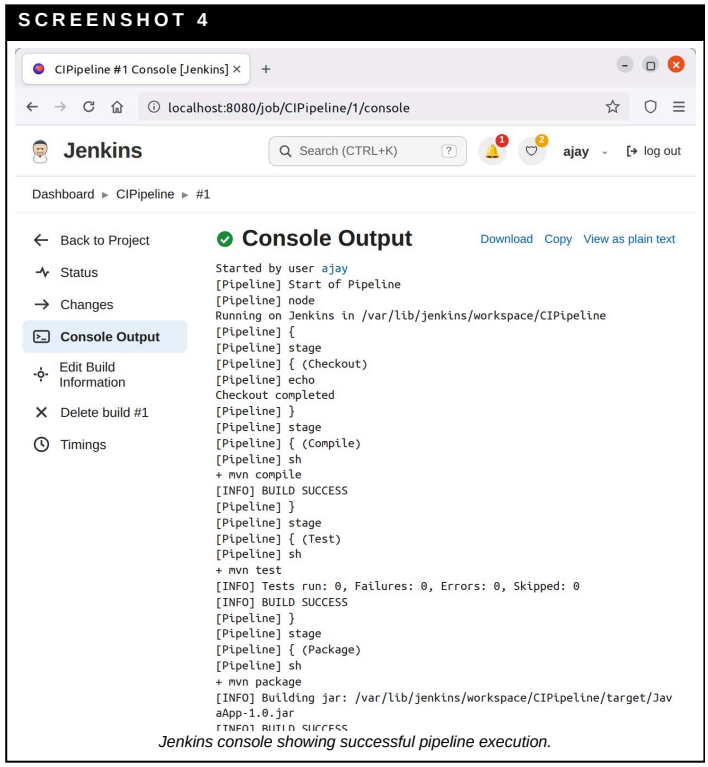
SCREENSHOT 3



Jenkins Pipeline stage view showing the executed stages.

15. Open Console Output and verify the build result.

SCREENSHOT 4



Jenkins console showing successful pipeline execution.

RESULT

A Jenkins Pipeline was configured and executed to automate the software build stages.

IMPORTANT PRACTICAL EXAM NOTE

For screenshots, use the actual terminal, browser, VirtualBox or Jenkins window where the operation was performed. The screenshot descriptions above indicate what to capture; they are not screenshots themselves.

For Experiments 8–10, Jenkins must be configured correctly, and the build user needs permission to execute the commands. For Experiment 9, a valid Maven `pom.xml` and Java source file must be present. For Experiment 10, the Maven project must be available in the Jenkins workspace for the `mvn` stages to execute.